چیت‌شیت جامع ساختار و زبان کامپیوتر

بر اساس اسلایدهای دکتر سربازی‌آزاد — جلسه‌های ۱ تا ۵ — برای امتحان کتاب‌باز (Open-book)

فرمول/قاعده | تعریف | مثال حل‌شده | تله/خطای رایج | پرسش تشریحی

۱. قواعد طلایی (فرمول‌های کلیدی) ۱
۲. حافظه و نمادگذاری ۱
۳. قالب دستور، کد عمل و مسیرهای داخلی ۱
۴. ثبات‌ها و چرخهٔ واکشی ۲
۵. روش‌های آدرس‌دهی (۸ روش) ۲
۶. ماشین‌های نمونه (جدول کامل کد عمل‌ها) ۲
۷. روش گام‌به‌گام ترجمه به کد ماشین ۳
۸. الگوهای آمادهٔ برنامه‌نویسی اسمبلی ۴
۹. اسمبلر، پیونددهنده، بارکننده و جابه‌جایی ۴
۱۰. ماشین پشته‌ای و نمادگذاری لهستانی ۵
۱۱. مبناها و تبدیل‌ها ۵
۱۲. اعداد علامت‌دار و سرریز ۵
۱۳. ممیز ثابت و ممیز شناور ۶
۱۴. استاندارد IEEE 754 ..... ۶
۱۵. کدهای دهدهی و کاراکتری ۷
۱۶. آدرس‌دهی نسبی، قطعه‌ای، صفحه‌ای و چندکاربره ۷
۱۷. فون‌نویمان، فلین و ابررایانه‌ها ۸
۱۸. پرسش‌های تشریحی با پاسخ کوتاه ۸
۱۹. تله‌های امتحانی و خطاهای اسلاید ۸
۲۰. داده‌های مرجع ۸

۱ قواعد طلایی (فرمول‌های کلیدی)

طول فیلدها و ثبات‌ها

- n بیت ⇐ 2^n حالت؛ M حالت ⇐ $\lceil \log_2 M \rceil$ بیت. (۳۲ حالت ۵ بیت، ۱۰۰۰ حالت ۱۰ بیت)
- فیلد کد عمل = $\lceil \log_2(\#ops) \rceil$؛ فیلد آدرس = $\lceil \log_2(\#units) \rceil$؛ فیلد ثبات = $\lceil \log_2(\#regs) \rceil$.
- $L_{MAR} = L_{PC}$ = طول آدرس حافظه = $\lceil \log_2(\#addressable\ units) \rceil$
- L_{MBR} = پهنای دسترسی (اگر گفته نشده = طول کلمه)
- L_{IR} = طول **بلندترین** قالب دستور؛ $L_{ACC} = L_R$ = طول کلمه.
- معمولاً: واحد آدرس‌پذیر ≤ کلمه ≤ پهنای دسترسی؛ اگر چیزی گفته نشده، هر سه برابرند.

آدرس‌ها و گام‌ها

- طول دستور بر حسب واحد = (بیت‌های واحد) / (بیت‌های دستور) ⇐ گام شمارندهٔ مکان (LC).
- کلمه بر حسب واحد = w/u ⇐ k = گام اشاره‌گر آرایه؛ آرایهٔ N عنصری = N·k واحد؛ آخرین عنصر = base + (N-1)·k.
- حافظهٔ بایتی + کلمهٔ ۱۶ بیتی ⇐ گام 2؛ کلمهٔ ۳۲ بیتی ⇐ گام 4؛ حافظهٔ نیبلی (۴ بیتی) + کلمهٔ ۱۶ بیتی ⇐ گام 4.
- **پرش نسبی:** $(addr_{jump} + len_{jump})$ = target - d؛ منفی ⇐ 2^k - |d| (۸ بیت: |d| - 100h)؛ بازهٔ ۸ بیتی 127+...-128.
- بانک در حافظهٔ k بانکی: bank = a mod k، row = ⌊a/k⌋. کلمهٔ هم‌تراز: a mod k = 0.
- قطعه‌ای: PA = (SR) + LA؛ صفحه‌ای: PA = (PR) ‖ LA؛ نسبی: EA = (PC) + d؛ شاخص‌دار: EA = addr + (Rx).

اعداد (n بیت)

#zeros	منفی کردن	Range	نمایش
1	–	0 … 2^n-1	بدون علامت
2	معکوس MSB	-(2^{n-1}-1) … 2^{n-1}-1	علامت-مقدار
2	معکوس همهٔ بیت‌ها: 2^n-1-x	-(2^{n-1}-1) … 2^{n-1}-1	مکمل یک
1	معکوس + 1: 2^n-x	-2^{n-1} … 2^{n-1}-1	مکمل دو
1	کد = مقدار + K	-K … 2^n-1-K	Excess-K

- **سرریز:** فقط وقتی دو عملوند هم‌علامت‌اند و نتیجه علامت مخالف دارد؛ در مکمل دو معادل $C_{in}(MSB) \neq C_{out}(MSB)$.
- مکمل یک: نقلی خروجی را به نتیجه اضافه کن (end-around)؛ مکمل دو: نقلی خروجی را دور بریز.
- ممیز شناور سفارشی: $(-1)^S \times 0.M \times 2^{E-bias}$؛ IEEE: $(-1)^S \times 1.F \times 2^{E-127}$ (single) / 2^{E-1023} (double).

۲ حافظه و نمادگذاری

تعریف‌ها

- **حافظه:** آرایهٔ خانه‌ها با آدرس 0 تا M-1. دسترسی با **MAR** (آدرس ← رمزگشا) و **MBR/MDR** (داده) و سیگنال‌های **Read/Write**.
- **نوشتن:** MAR ← a; MBR ← data; Write. **خواندن:** MAR ← a; Read ⇒ MBR ← $(M_{(MAR)})$.
- **واحد آدرس‌پذیر:** کوچک‌ترین گروه بیتی با آدرس یکتا (معمولاً ۸ بیت). **کلمه:** طول عملوندهای پردازش‌شده توسط CPU (ماشین n بیتی). **پهنای دسترسی:** بیت‌های جابه‌جاشده در یک دسترسی.
- **هم‌تراز:** آدرس کلمه مضرب طول آن ⇐ یک دسترسی ☺ . **ناهم‌تراز:** هر آدرسی ⇐ ممکن است دو دسترسی ☹ (وقتی کلمه از مرز سطر بگذرد).
- **Big Endian:** بایت پرارزش در آدرس کوچک‌تر (IBM 360/370 و ماشین‌های درس). **Little Endian:** بایت کم‌ارزش در آدرس کوچک‌تر (Intel x86، RISC-V).

12345678h @100h	100	101	102	103
Big Endian	12	34	56	78
Little Endian	78	56	34	12

نمادگذاری و نمونه

M_a = خانهٔ a؛ (M_a) = محتوای آن؛ $M_{(M_a)}$ = خانه‌ای که آدرسش محتوای a است؛ A ← B = نوشتن. پرانتز دور ثبات = محتوای ثبات.

حافظهٔ اسلاید (آدرس 0..15):

10 11 21 31 14 1010 61 17 81 91 110 111 1120 131 114 115

- $(M_9) = 91$؛ $M_{(M_0)} = M_{10}$؛ $(M_{(M_1)}) = (M_{11}) = 111$
- $M_{10} \leftarrow 10 + 114 = 124 \Rightarrow M_{(M_0)} \leftarrow (M_0) + (M_{(M_4)})$

حافظهٔ ۴ بانکی (بایتی، کلمهٔ ۳۲ بیتی، 2^{16} بایت)

MAR ۱۶ بیت: ۱۴ بیت پرارزش ⇐ رمزگشای سطر (2^{14} کلمه)، ۲ بیت کم‌ارزش ⇐ انتخاب بانک؛ MBR ۳۲ بیت (۴×۸). مثال: 1235h = ...0100 1000 1101|01 ⇒ بانک 1، سطر 048Dh = 1165. کلمه از آدرس 5 ⇒ بایت‌های 5..8 ⇒ سطرهای 1 و 2 ⇒ دو دسترسی.

۳ قالب دستور، کد عمل و مسیرهای داخلی

دستور = **عمل (Opcode)** + **عملوندها** (فاعل همیشه CPU است و ذکر نمی‌شود).

ماشین n آدرسه:

Opcode	Op 1	...	Op n

کد عمل می‌تواند نشان دهد: ۱) نوع عمل ۲) تعداد عملوندها ۳) طول دستور (اگر چند قالب) ۴) نوع عملوندها (صحیح/حقیقی/علامت‌دار) ۵) طول عملوندها ۶) روش آدرس‌دهی.

کد عمل گسترش‌پذیر (ماشین ۵): ۴ بیت اول 1111 ⇐ ۶ بیت بعد هم جزو کد عمل است.

مسیرهای داخلی ماشین

۱	واکشی دستور: حافظه ← CU	۴	داده‌ٔ بی‌واسطه: ALU ← CU
۲	دسترسی داده: حافظه ⇄ ALU	۵	I/O از طریق CPU
۳	فرمان/وضعیت: ALU ⇄ CU	۶	I/O با DMA (بدون CPU)

نمونه: اجرای `sub num1,num2` در ماشین ۱ ⇒ مسیر ۱ (واکشی)، ۲ (خواندن M_{15})، ۲ (خواندن M_{10})، ۳ (فرمان تفریق)، ۲ (نوشتن M_{15}).

رمزگشایی و کدگذاری در ماشین ۱ (op 2 | a1 7 | a2 7)

`sub num1,num2` ⇐ `01 0001111 0001010` ، a1=15، a2=10 ⇐ sub ، $M_{15} \leftarrow (M_{15}) - (M_{10})$ =
`add 3,100` ⇐ `00 0000011 1100100` ؛ `01E4h` = `A1B5h`
$M_{67} \leftarrow (M_{53})$ ⇐ `10 1000011 0110101` =.

۴ ثبات‌ها و چرخهٔ واکشی

- **ثبات n بیتی** = n فلیپ‌فلاپ با کنترل گروهی (کلاک مشترک).
- **PIPO** (ورود و خروج موازی؛ ثبات‌های معمولی)، **SIPO** (سریال← موازی؛ گیرنده)، **PISO** (موازی← سریال؛ با MUX و Load/Shift')، **SISO** (خط تأخیر n کلاکی).
- **یکپارچه (Register File)**: مثل حافظهٔ کوچک با رمزگشا ⇐ هر لحظه یک ثبات.
  مجزا (Isolated): خطوط جدا ⇐ دسترسی موازی.
- **آدرس‌پذیر** (R0–R15 در AX/BX ،IBM 360/... در 8086) / **غیرآدرس‌پذیر** (MAR، MBR، IR، PC).
- **همه‌منظوره** (R0–R15 در 360) / **خاص‌منظوره** (CX شمارش حلقه، DX ضرب/تقسیم I/O در 8086).
- **MAR** آدرس سیکل حافظه؛ **MBR** دادهٔ مبادله؛ **IR** دستور واکشی‌شده؛ **PC** آدرس دستور **بعدی**.

واکشی: MAR ← (PC); MBR ← $(M_{(MAR)})$; IR ← (MBR); PC ← (PC) + len ⇐
رمزگشایی ⇐ خواندن عملوندها ⇐ اجرا ⇐ نوشتن نتیجه. PC هنگام اجرا به دستور بعد اشاره می‌کند.

SIPO با ورودی 1,0,1,1: `1101 → 1010 → 0100 → 1000` (Q3..Q0). **PISO** با 1011: خروجی سریال 1,1,0,1 (اول LSB).

۵ روش‌های آدرس‌دهی (۸ روش)

#	روش	عملوند / آدرس مؤثر	دسترسی حافظه برای عملوند	مثال
۱	**مستقیم** (ثباتی/حافظه‌ای)	(R) / (M_a)	1 / 0	mov ax,array ; L 2,NUM
۲	**غیرمستقیم** (ثباتی/حافظه‌ای)	$(M_{(R)})$ / $(M_{(M_a)})$	2 / 1	L 2,0(3) ; call far ptr [p1]
۳	**بی‌واسطه**	خود عملوند در IF	0	sub cx,33 ; addi $s0,255
۴	**شاخص‌دار**	$(M_{a+(Rx)})$	1	sub ax,array[bx]
۵	**ضمنی**	در تعریف دستور (ACC، cx، ...)	—	loop addr ; MR 2,4
۶	**نسبی**	EA = (PC) + d	(پرش)	jcxz loop_addr
۷	**قطعه‌ای**	PA = (SR) + LA	+ یک جمع	جابه‌جایی بدون تغییر کد
۸	**صفحه‌ای**	PA = (PR) ‖ LA	بدون جمع	الحاق بیت‌ها

- `loop addr` (8086): if (cx)≠0: PC ← addr ; cx ← (cx)-1 .`MR 2,4` (360): R3 ⇐ R2:R3 ← (R3)×(R4) ضمنی.
- غیرمستقیم چندسطحی: یک سطح $M_{(M_a)}$، دو سطح $M_{(M_{(M_a)})}$؛ بیشتر ماشین‌ها یک سطح، تعداد کمی دو سطح.
- تقریباً همهٔ ماشین‌ها: مستقیم، غیرمستقیم، بی‌واسطه؛ بیشتر ماشین‌ها: شاخص‌دار.

مثال: $(M_{200})=300$, $(M_{300})=400$, $(M_{400})=7$, (R3)=300

روش	EA	Operand
مستقیم a=200	200	300
غیرمستقیم ۱ سطحی	300	400
غیرمستقیم ۲ سطحی	400	7
ثباتی مستقیم R3	—	300
ثباتی غیرمستقیم (R3)	300	400

۶ ماشین‌های نمونه (جدول کامل کد عمل‌ها)

ماشین ۱ — دوآدرسه

۱۲۸ خانهٔ ۱۶ بیتی؛ ۱۶ بیت = | op 2 | a1 7 | a2 7 | ؛ $L_{MAR}=7$, $L_{MBR}=16$

Opcode	Mnemonic	Operation
00	`add a1,a2`	$M_{a1} \leftarrow (M_{a1}) + (M_{a2})$
01	`sub a1,a2`	$M_{a1} \leftarrow (M_{a1}) - (M_{a2})$
10	`mov a1,a2`	$M_{a1} \leftarrow (M_{a2})$
11	`nop`	no operation

ماشین ۲ — دوآدرسه (بدون پرش!)

2^{16}×8 بیت؛ کلمه ۱۶؛ | op 8 | a1 16 | a2 16 | = ۵ بایت؛
MAR=16, MBR=16, IR=40, PC=16

Opcode	Mnemonic	Operation
00	`mov a1,a2`	$M_{a1} \leftarrow (M_{a2})$
01	`add a1,a2`	$M_{a1} \leftarrow (M_{a1}) + (M_{a2})$
02	`sub a1,a2`	$M_{a1} \leftarrow (M_{a1}) - (M_{a2})$
03	`and a1,a2`	$M_{a1} \leftarrow (M_{a1}) \wedge (M_{a2})$
04	`or a1,a2`	$M_{a1} \leftarrow (M_{a1}) \vee (M_{a2})$
05	`xor a1,a2`	$M_{a1} \leftarrow (M_{a1}) \oplus (M_{a2})$

ماشین ۳ — دوآدرسه + غیرمستقیم + پرش شرطی

مثل ماشین ۲؛ کلمهٔ ۱۶ بیتی ناهم‌تراز، Big Endian؛
MAR=16, MBR=16, IR=40, PC=16. در پرش‌ها a1 = متغیر شرط، a2 = مقصد. پرش غیرشرطی و بی‌واسطه ندارد.

Opcode	Mnemonic	Operation
00	`mov a1,a2`	$M_{a1} \leftarrow (M_{a2})$
01	`mov a1,(a2)`	$M_{a1} \leftarrow (M_{(M_{a2})})$
02	`mov (a1),a2`	$M_{(M_{a1})} \leftarrow (M_{a2})$
03	`add a1,a2`	$M_{a1} \leftarrow (M_{a1}) + (M_{a2})$
04	`sub a1,a2`	$M_{a1} \leftarrow (M_{a1}) - (M_{a2})$
05	`add a1,(a2)`	$M_{a1} \leftarrow (M_{a1}) + (M_{(M_{a2})})$
06	`sub a1,(a2)`	$M_{a1} \leftarrow (M_{a1}) - (M_{(M_{a2})})$
80	`jnz a1,a2`	if $(M_{a1}) \neq 0$: PC ← a2
81	`jz a1,a2`	if $(M_{a1}) = 0$: PC ← a2
82	`jneg a1,a2`	if $(M_{a1}) < 0$: PC ← a2
83	`jpos a1,a2`	if $(M_{a1}) > 0$: PC ← a2

ماشین ۴ — دوآدرسه، ۱۶ ثبات، حافظهٔ ۴ بیتی

2^{12}×4 بیت؛ کلمه ۱۶ = **۴ واحد**؛ ناهم‌تراز، BE؛ R0..R15؛ هر رقم هگز = یک واحد = یک فیلد.
MAR=12, MBR=16, IR=24, PC=12, R=16

Opcode	Mnemonic	Operation
Format I: op4 | r1 4 | r2 4 (3 units)		
0	`mov r1,r2`	r1 ← (r2)
1	`mov r1,(r2)`	r1 ← $(M_{(r2)})$
2	`mov (r1),r2`	$M_{(r1)}$ ← (r2)
3	`add r1,r2`	r1 ← (r1)+(r2)
4	`sub r1,r2`	r1 ← (r1)-(r2)
Format II: op4 | r 4 | addr 12 (5 units)		
5	`mov r,addr`	r ← (M_{addr})
6	`mov addr,r`	M_{addr} ← (r)
7	`mov r,(addr)`	r ← $(M_{(M_{addr})})$
8	`mov (addr),r`	$M_{(M_{addr})}$ ← (r)
9	`jz r,addr`	if (r)=0: PC ← addr
A	`jnz r,addr`	if (r)≠0: PC ← addr
B	`jneg r,addr`	if (r)<0: PC ← addr
C	`jpos r,addr`	if (r)>0: PC ← addr
Format III: op4 | r 4 | data 16 (6 units)		
D	`mov r,#data`	r ← data
E	`add r,#data`	r ← (r)+data
F	`sub r,#data`	r ← (r)-data

ماشین ۵ — ۲/۳ آدرسه، ۶۴ ثبات

2^{16}×8 بیت؛ کلمه ۱۶؛ ناهم‌تراز، BE؛ R0..R63؛ آدرس‌دهی: ثباتی، مستقیم، بی‌واسطه، شاخص‌دار. MAR=16, MBR=16, IR=32, PC=16, R=16

Opcode	Mnemonic	Operation
Format I: op4 | r1 6 | r2 6 (2 bytes) — hex = op·1000h + r1·40h + r2		
0	`mov r1,r2`	r1 ← (r2)
1	`add r1,r2`	r1 ← (r1)+(r2)
2	`sub r1,r2`	r1 ← (r1)-(r2)
3	`and r1,r2`	r1 ← (r1)∧(r2)
4	`or r1,r2`	r1 ← (r1)∨(r2)
5	`xor r1,r2`	r1 ← (r1)⊕(r2)
Format II: op4 | r1 6 | r2 6 | addr 16 (4 bytes)		
6	`mov r1,addr(r2)`	r1 ← ($M_{addr+(r2)}$)
7	`mov addr(r2),r1`	$M_{addr+(r2)}$ ← (r1)
8	`mov r1,r2,addr`	r1 ← (r2); r2 ← (M_{addr})
9	`mov addr,r1,r2`	M_{addr} ← (r1); r1 ← (r2)
A	`je r1,r2,addr`	if (r1)=(r2): PC ← addr
B	`jne r1,r2,addr`	if (r1)≠(r2): PC ← addr
C	`jl r1,r2,addr`	if (r1)<(r2): PC ← addr
D	`jh r1,r2,addr`	if (r1)>(r2): PC ← addr
E	`call r1,addr(r2)`	r1 ← (PC); PC ← addr+(r2)
Format III: op10 | r 6 | data 16 (4 bytes) — first word = F000h + sub·40h + r		
1111 000000	`mov r,#data`	r ← data
1111 000001	`add r,#data`	r ← (r)+data
1111 000010	`sub r,#data`	r ← (r)-data
1111 000011	`and r,#data`	r ← (r)∧data
1111 000100	`or r,#data`	r ← (r)∨data
1111 000101	`xor r,#data`	r ← (r)⊕data

با r1=r2: `mov r,r,addr` = بار کردن از حافظه، `mov addr,r,r` = ذخیره. بازگشت به OS: `call R0,0(R63)` (R63 = آدرس بازگشت). `xor R0,R0` = صفر کردن.

ماشین تک‌آدرسه (انباره‌ای)

2^{16}×4 بیت؛ کلمه ۱۶ = ۴ واحد؛ ناهم‌تراز، BE؛ [op 4 | addr/data 16] = ۵ واحد؛ MAR=16, MBR=16, ACC=16, IR=20, PC=16

Opcode	Mnemonic	Operation
0	`load #data`	ACC ← data
1	`add #data`	ACC ← (ACC)+data
2	`load addr`	ACC ← (M_{addr})
3	`store addr`	M_{addr} ← (ACC)
4	`add addr`	ACC ← (ACC)+(M_{addr})
5	`sub addr`	ACC ← (ACC)-(M_{addr})
6	`and addr`	ACC ← (ACC)∧(M_{addr})
7	`or addr`	ACC ← (ACC)∨(M_{addr})
8	`xor addr`	ACC ← (ACC)⊕(M_{addr})
9	`load (addr)`	ACC ← ($M_{(M_{addr})}$)
A	`store (addr)`	$M_{(M_{addr})}$ ← (ACC)
B	`j addr`	PC ← addr
C	`jz addr`	if (ACC)=0: PC ← addr
D	`jnz addr`	if (ACC)≠0: PC ← addr
E	`jn addr`	if (ACC)<0: PC ← addr
F	`jp addr`	if (ACC)>0: PC ← addr

ماشین ۶ — صفرآدرسه (پشته‌ای)

2^{32}×8 بیت؛ کلمه ۳۲؛ آدرس‌دهی: ضمنی، مستقیم، بی‌واسطه؛ SP به اولین خانهٔ **خالی** اشاره می‌کند. *در اسلاید کد jn اشتباهاً 08 (= j) آمده است.

Opcode	Mnemonic	Operation
Format I: op 8 | addr/data 32 (5 bytes)		
00	`push #data`	$Stack_{(SP)}$ ← data; SP++
01	`push addr`	$Stack_{(SP)}$ ← (M_{addr}); SP++
02	`pop addr`	SP--; M_{addr} ← ($Stack_{(SP)}$)
Format II: op 8 (1 byte)		
03	`add`	$Stack_{(SP)-2}$ ← ($Stack_{(SP)-2}$)+($Stack_{(SP)-1}$); SP--
04	`sub`	$Stack_{(SP)-2}$ ← ($Stack_{(SP)-2}$)-($Stack_{(SP)-1}$); SP--
05	`swap`	$Stack_{(SP)-1}$ ↔ $Stack_{(SP)-2}$
06	`sp++`	SP ← (SP)+1
07	`sp--`	SP ← (SP)-1
08	`j`	PC ← ($Stack_{(SP)-1}$); SP--
09	`jz`	if ($Stack_{(SP)-2}$)=0: PC ← ($Stack_{(SP)-1}$); SP -= 2
0A	`jnz`	if ($Stack_{(SP)-2}$)≠0: PC ← ($Stack_{(SP)-1}$); SP -= 2
0B*	`jn`	if ($Stack_{(SP)-2}$)<0: PC ← ($Stack_{(SP)-1}$); SP -= 2

ماشین ۷ — تک‌آدرسه با پرش نسبی

2^{32}×8 بیت؛ کلمه ۳۲ (مکمل دو)، ناهم‌تراز، BE؛ آدرس‌دهی: مستقیم، غیرمستقیم، بی‌واسطه، نسبی. MAR=32, MBR=32, ACC=32, IR=40, PC=32

Opcode	Mnemonic	Operation
Format I: op 8 | addr/data 32 (5 bytes)		
00	`load addr`	ACC ← (M_{addr})
01	`load (addr)`	ACC ← ($M_{(M_{addr})}$)
02	`store addr`	M_{addr} ← (ACC)
03	`store (addr)`	$M_{(M_{addr})}$ ← (ACC)
04	`add addr`	ACC ← (ACC)+(M_{addr})
05	`sub addr`	ACC ← (ACC)-(M_{addr})
06	`addi data`	ACC ← (ACC)+data
07	`subi data`	ACC ← (ACC)-data
08	`jmpl addr`	PC ← addr (absolute)
Format II: op 8 | displacement 8 (2 bytes) — PC = next instr.		
80	`jnz addr`	if (ACC)≠0: PC ← (PC)+d
81	`jz addr`	if (ACC)=0: PC ← (PC)+d
82	`jneg addr`	if (ACC)<0: PC ← (PC)+d
83	`jpos addr`	if (ACC)≥0: PC ← (PC)+d
84	`jmp addr`	PC ← (PC)+d

۷ روش گام‌به‌گام ترجمه به کد ماشین

1. **طول هر قالب** را بر حسب واحد آدرس‌پذیر حساب کن (بایت یا نیبل).
2. **گذر ۱:** LC = مقدار `org`. برای هر خط: آدرس = LC؛ برچسب را در جدول نمادها ثبت کن؛ LC += طول (دستور یا `dw` = یک کلمه؛ `dw N dup(?)` = N کلمه). داده‌ها معمولاً بعد از کد می‌آیند.
3. **گذر ۲:** برای هر دستور: کد عمل ‖ فیلدها (آدرس از جدول نمادها) ⇐ بیت‌ها را پشت‌سرهم بنویس و چهارتا چهارتا هگز کن (مخصوصاً وقتی فیلدها مضرب ۴ نیستند، مثل ماشین ۵).
4. **پرش نسبی:** d = target − next؛ منفی را مکمل دو کن؛ اگر در بازه نیست از پرش مطلق (`jmpl`) استفاده کن.
5. مقدار اولیهٔ داده‌ها: `dw array` = آدرس array؛ `dw ?` = ????؛ عدد منفی = مکمل دو (FFFF/FFFFFFFF = -1).
6. در پایان برنامه کنترل را به OS برگردان (وگرنه CPU داده‌ها را به عنوان دستور اجرا می‌کند).

نمونهٔ کامل: جمع آرایهٔ ۱۰۰ کلمه‌ای در ماشین ۳ (غیرمستقیم)

```
addr  machine code   source
                     org  100h
0100  05 0116 0114   loop: add sum,(arrptr)
0105  03 0114 011A         add arrptr,two
010A  04 0118 011A         sub count,two
010F  80 0118 0100         jnz count,loop
0114  011C           arrptr: dw array
0116  0000           sum:    dw 0
0118  00C8           count:  dw 200   ; 100 iters
011A  0002           two:    dw 2
011C  ????…          array:  dw 100 dup(?)
```

نمونهٔ کدگذاری در هر ماشین (همه آزموده‌شده)

M	Instruction	Code	How
4	mov R0,#041h	D00041	D|0|0041 (1 hex = 1 field)
4	mov R10,(R0)	1A0	1|A|0
4	add R10,R11	3AB	3|A|B
4	jnz R5,018h	A5018	A|5|018
5	mov R3,A(R1)	60C1 0120	0110|000011|000001
5	add R0,R3	1003	1000h+0·40h+3
5	mov R2,#200	F002 00C8	F000h+0·40h+2
5	add R1,#2	F041 0002	F000h+1·40h+1
5	jne R1,R2,108h	B042 0108	B000h+1·40h+2
5	add R62,Rk	1F80h+k	0001|111110|k
5	call R0,0(R63)	E03F 0000	E000h+0+3Fh
1A	load (0055)	90055	9|0055
1A	add #-1	1FFFF	-1 = FFFF
6	push x (5Dh)	01 0000005D	5 bytes
6	add / jnz / j	03 / 0A / 08	1 byte
7	jpos continue	83 34	@128h → 15Eh: 15E-12A
7	jmp loop	84 A2	@15Ch → 100h: -5Eh
7	jnz loop	80 96	@168h → 100h: -6Ah

1A = ماشین تک‌آدرسه. در ماشین ۷ پرش با فاصلهٔ کمتر از 128- (مثلاً 140-) ممکن نیست ⇐ jmpl.

۸ الگوهای آمادهٔ برنامه‌نویسی اسمبلی

رهنمودها (Directives) — کد ماشین تولید نمی‌کنند

یک کلمه با مقدار v	lbl: dw v	ادامه از آدرس X	org X
چند مقدار پشت‌سرهم	dw 0,1,18 dup(?)	کلمهٔ نامعلوم N	dw N dup(?)
واحد بعد از برچسب k	label+k	پایان برنامهٔ مبدأ	end

حلقهٔ شمارشی و اشاره‌گر (ماشین ۳)

```
loop: add  sum,curr     ; body ...
      sub  count,one    ; count--
      jnz  count,loop
      jnz  one,OS_ret   ; "always" jump
count: dw N
one:   dw 1           ; no immediates!
```

مجموع فیبوناچی متناوب: هر دور add sum,curr / sub sum,next / add curr,next / add next,curr با count=50 (۱۰۰ جمله).

بدون غیرمستقیم: کد خودتغییرده (ماشین ۳)

```
loop: add  sum,array    ; addr2 field = loop+3
      add  loop+3,two   ; patch: next element
      sub  count,two
      jnz  count,loop
```

فیلد addr2 از بایت loop+3 شروع می‌شود (۱ بایت کد عمل + ۲ بایت addr1). **در اجرای دوباره باید فیلد را دوباره مقداردهی کرد.**

ثبات‌ها + غیرمستقیم ثباتی (ماشین ۴: C = A + B)

```
      mov R0,#A ; mov R1,#B ; mov R2,#C
      mov R5,#10
loop: mov R10,(R0)
      mov R11,(R1)
      add R10,R11
      mov (R2),R10
      add R0,#4 ; add R1,#4 ; add R2,#4
      sub R5,#1
      jnz R5,loop      ; A=041h B=069h C=091h
```

شاخص‌دار (ماشین ۵: جمع آرایه)

```
      xor  R0,R0         ; sum = 0
      mov  R1,R0         ; index = 0
      mov  R2,#200       ; end index
loop: mov  R3,A(R1)
      add  R0,R3
      add  R1,#2
      jne  R1,R2,loop
      mov  sum,R0,R0     ; store
      call R0,0(R63)     ; return to OS
```

انباره‌ای (تک‌آدرسه: ۲۰ جملهٔ فیبوناچی)

```
loop: load (p1) ; store sum
      load (p2) ; add sum
      store (p3)          ; A[i+2]=A[i]+A[i+1]
      load p1 ; add #4 ; store p1
      add #4 ; store p2 ; add #4 ; store p3
      load counter ; add #-1 ; store counter
      jnz loop            ; tests ACC
      j OS_return_addr
p1: dw array ; p2: dw array+4 ; p3: dw array+8
counter: dw 18 ; array: dw 0,1,18 dup(?)
```

کد ماشین: داده‌ها از 0055h (۱۷ دستور × ۵ واحد)؛ p1=0055، p2=0059، p3=005D، counter=0061، sum=0065، array=0069.

پشته‌ای (ماشین ۶)

```
push x ; push y ; add ; pop x   ; x = x+y
sp++                            ; x back on top
push counter ; push #1 ; sub
pop counter ; sp++              ; [counter]
push #loop ; jnz                ; cond, then target
push #OS_return_addr ; j
```

ترفند: pop x داده را پاک نمی‌کند ⇒ ذخیره بدون برداشتن (dup ندارد) = pop x / sp++.
اتحاد: $\Sigma F_{0..n-1} = F_{n+1} - 1$ ⇒ برنامهٔ کوتاه‌تر.

ترفندهای کوچک

- **جابه‌جایی بدون متغیر کمکی:** xor x,y / xor y,x / xor x,y.
- **صفر کردن:** xor R,R یا sub x,x. **دو برابر:** add r,r. **ضرب در ۴:** دو بار add.
- **پرش غیرشرطی بدون دستور مخصوص:** پرش شرطی روی متغیری که همیشه ≠ 0 است.
- **ثابت‌ها بدون بی‌واسطه:** در حافظه تعریف کن (one: dw 1)؛ با شمارندهٔ 200 و گام 2 یک ثابت کمتر لازم است.
- **نگهبان (sentinel):** مقدار ویژه قبل/بعد آرایه برای حذف یک شرط (مرتب‌سازی Gnome).

بازگشت به سیستم‌عامل				
M3: jnz one,OS	M5: call R0,0(R63)	1-addr: j OS	M6: push #OS ; j	M7: jmpl OS

مرتب‌سازی Gnome (تنها مرتب‌سازی غیربازگشتی با یک حلقه)

```
i := 1;
while i < n do
  if A[i] > A[i+1] then
    begin swap(A[i],A[i+1]);
          if i > 1 then i := i-1; end
  else i := i+1;
```

در ماشین ۷: ACC = A[i+1] - A[i]، jpos continue (مرتب)؛ نگهبان 80000000h قبل از آرایه؛ پایان وقتی p = array + 396. **اگر A[1] ≥ 0 تفریق با نگهبان سرریز می‌کند ⇒ نگهبان C0000000h یا بررسی p = array.**

۹ اسمبلر، پیونددهنده، بارکننده و جابه‌جایی

روند: برنامهٔ سطح بالا ⇐ **کامپایلر** ⇐ اسمبلی ⇐ **اسمبلر** ⇐ فایل شیء (+ فایل فهرست) ⇐ **پیونددهنده** (+ کتابخانه‌ها) ⇐ فایل اجرایی ⇐ **بارکننده** ⇐ برنامه در حافظه.

اسمبلر دوگذره: گذر ۱ با شمارندهٔ مکان آدرس برچسب‌ها را در **جدول نمادها** ثبت می‌کند؛ گذر ۲ کد ماشین را تولید می‌کند. دلیل: **ارجاع رو به جلو** (استفاده از برچسبی که بعداً تعریف شده).

فایل شیء: ۱) سرآیند (اندازه و محل بخش‌ها) ۲) بخش متن (دستورها) ۳) بخش داده ۴) اطلاعات جابه‌جایی (وابسته به آدرس مطلق) ۵) جدول نمادها (برچسب‌های خارجی + ارجاع‌های حل‌نشده) ۶) اطلاعات اشکال‌زدایی.

پیونددهنده (۳ کار): ۱) یافتن روال‌های کتابخانه‌ای (printf، scanf، …) ۲) تعیین محل حافظهٔ هر ماژول و **جابه‌جایی** ارجاع‌های مطلق ۳) حل ارجاع‌های میان فایل‌ها (مثلاً jal ??? ⇐ jal printf).

بارکننده (۶ گام): ۱) خواندن سرآیند ۲) ساخت فضای آدرس ۳) کپی دستورها و داده‌ها ۴) کپی آرگومان‌ها روی پشته ۵) مقداردهی ثبات‌ها و SP ۶) پرش به روال آغازین که main را صدا می‌زند.

جابه‌جایی برنامه (Relocating loader)

اگر برنامه برای `org 100h` اسمبل شده ولی از `50100h` بار شود ⇒ همهٔ **آدرس‌ها** 50000h+ می‌شوند. **Q1:** دادهٔ بی‌واسطه‌ای که عدد است (`addi 4`) نباید تغییر کند ولی آن که آدرس است (`subi array+396`، `mov R0,#A`) باید تغییر کند ⇐ فقط اسمبلر می‌داند (رکورد جابه‌جایی). **Q2:** اشاره‌گرهای داخل داده (`p: dw array`) هم باید جابه‌جا شوند؛ آدرس‌های ساخته‌شده در زمان اجرا اصلاً قابل اصلاح نیستند.

راه‌حل: آدرس‌دهی قطعه‌ای ⇐ کد دست‌نخورده، بارکننده فقط ثبات قطعه را `00050000h` می‌کند.

۱۰ ماشین پشته‌ای و نمادگذاری لهستانی

- پشتهٔ سخت‌افزاری LIFO داخل CPU؛ SP به جای خالی بعدی اشاره می‌کند (SP=0 یعنی خالی).
- push x: $Stack_{(SP)} \leftarrow x$; $SP \leftarrow SP+1$ pop: $SP \leftarrow SP-1$; return $Stack_{(SP)}$
- تابع: pop t1; pop t2; t3 ← func(t1,t2); push t3. تفریق/تقسیم: **پایینی - بالایی** (اول a بعد b ⇒ a-b).
- مثال اسلاید: `push #-10; push Num(100); push #77; pop Num` ⇒ SP = 2، Num = 77، پشته [-10, 100]؛ سپس `add` ⇒ [90]، SP = 1.

میانوندی ← پسوندی (لهستانی)

روش اسلاید: ۱) پرانتزگذاری کامل ۲) از درونی‌ترین: پرانتز راست ← عملگر، پرانتز چپ حذف.

روش پشته‌ای (ایستگاه قطار): عملوند ⇐ خروجی؛ عملگر ⇐ تا وقتی بالای پشته اولویت ≤ دارد pop به خروجی، سپس push؛ «(» push؛ «)» ⇐ pop تا «(»؛ پایان ⇐ pop همه. اولویت: - + < / > *، هم‌اولویت‌ها چپ‌به‌راست.

محاسبه: چپ به راست؛ عملوند ⇐ push؛ عملگر ⇐ دو تا pop، نتیجه push.
پیشوندی: عملگر قبل از عملوندها (از راست بخوان).

مزایا: بدون پرانتز و بدون نگرانی اولویت؛ مناسب ماشین پشته‌ای. **عیب:** برای انسان ناخوانا.

مثال‌های حل‌شده

`(x+y-10-t/d+10*s-z) / (20*t-z/t+100*w)`

⇐ `xy+10-td/-10s*+z-20t*zt/-100w*+/` (چپ‌به‌راست استاندارد).

پرانتزگذاری اسلاید `…10s*z-+…` هم درست و هم‌ارزش است.

ردگیری (SP اولیه 2): بیشینهٔ SP = 6 (۴ خانهٔ آزاد لازم)؛ در پایان `pop Result` و SP = 2.

Infix	Postfix	Prefix
a+b*c-d	abc*+d-	-+a*bcd
(a+b)*(c-d)/e	ab+cd-*e/	/*+ab-cde
a*b+c*d-e	ab*cd*+e-	-+*ab*cde
a-(b-c)	abc--	-a-bc

`ab+cd-*e/` با a=2,b=3,c=7,d=1,e=5:
[6] → [30,5] → [6] → [5,7,1] → [5,6] → [5,7] → [5] → [2,3] → [2].

۱۱ مبناها و تبدیل‌ها

- **مبنای R ← دهدهی:** $\Sigma_{k=-m}^{n-1} a_k R^k$. مثال: $1101.101_2=13.625$، $572.6_8=378.75$، $2A.8_{16}=42.5$، $341.24_5=96.56$.
- **دهدهی ← R (صحیح):** تقسیم‌های متوالی بر R؛ اولین باقی‌مانده = LSB. $43 = 101011_2$، $156 = 9C_{16}$.
- **دهدهی ← R (کسری):** ضرب‌های متوالی در R؛ اولین رقم صحیح = MSB. $0.3125 = 0.0101_2$، $0.8125 = 0.64_8$، $0.1 = 0.0(0011)\ldots_2$ (متناوب!).
- **میان توان‌های ۲:** هر رقم را با i بیت بنویس، از ممیز گروه‌های ۳ بیتی بساز: $F2A_{16} = 111\ 100\ 101\ 010_2$؛ $7452_8 = 3AF.C_{16}$؛ $10110111_2 = 2313_4$.
- نمادها: C `0xF2`؛ Verilog `8'hF0 = 8'd240 = 8'b11110000`.

Dec	Bin	Oct	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

Dec	Bin	Oct	Hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

توان‌های ۲

n	2^n	n	2^n
0	1	16	65,536
1	2	17	131,072
2	4	18	262,144
3	8	19	524,288
4	16	20	1,048,576
5	32	21	2,097,152
6	64	22	4,194,304
7	128	23	8,388,608
8	256	24	16,777,216
9	512	25	33,554,432
10	1,024	26	67,108,864
11	2,048	27	134,217,728
12	4,096	28	268,435,456
13	8,192	29	536,870,912
14	16,384	30	1,073,741,824
15	32,768	31	2,147,483,648
16	65,536	32	4,294,967,296

n	2^n
-1	0.5
-2	0.25
-3	0.125
-4	0.0625
-5	0.03125
-6	0.015625
-7	0.0078125
-8	0.00390625
-9	0.001953125
-10	0.0009765625
-11	0.00048828125
-12	0.000244140625

2^{10}=K, 2^{20}=M, 2^{30}=G, 2^{40}=T؛ $\log_2 10 \approx 3.32$؛ هر رقم هگز = ۴ بیت، هر رقم اکتال = ۳ بیت.

۱۲ اعداد علامت‌دار و سرریز

Value	Sign-Mag	1s	2s	Excess-8	Excess-7
+8	–	–	–	–	1111
+7	0111	0111	0111	1111	1110
+6	0110	0110	0110	1110	1101
+5	0101	0101	0101	1101	1100
+4	0100	0100	0100	1100	1011
+3	0011	0011	0011	1011	1010
+2	0010	0010	0010	1010	1001
+1	0001	0001	0001	1001	1000
+0	0000	0000	0000	1000	0111
-0	1000	1111	–	–	–
-1	1001	1110	1111	0111	0110
-2	1010	1101	1110	0110	0101
-3	1011	1100	1101	0101	0100
-4	1100	1011	1100	0100	0011
-5	1101	1010	1011	0011	0010
-6	1110	1001	1010	0010	0001
-7	1111	1000	1001	0001	0000
-8	–	–	1000	0000	–

علامت-مقدار (SM)

جمع A+B: اگر $A_s=B_s \Rightarrow (A_s \mid A_m+B_m)$؛ وگرنه بزرگ منهای کوچک با علامت بزرگ‌تر:

$A_s=1, A_m>B_m \rightarrow (1 \mid A_m-B_m)$	$A_s=1, A_m<B_m \rightarrow (0 \mid B_m-A_m)$
$A_s=0, A_m>B_m \rightarrow (0 \mid A_m-B_m)$	$A_s=0, A_m<B_m \rightarrow (1 \mid B_m-A_m)$

تفریق: شرط اول $A_s \neq B_s \Rightarrow (A_s \mid A_m+B_m)$، بقیه همان. ضرب/تقسیم: $(A_s \oplus B_s \mid A_m \times B_m)$. **جمع/تفریق SM پیچیده‌تر و پرسخت‌افزارتر از مکمل‌هاست.**

مثال‌ها (۸ بیت): `00110100` = +52، `10010011` = -19؛ منفی `00100001` = `10100001`.

مکمل یک و مکمل دو

- مکمل یک: $-x = 2^n - x - 1$؛ 11110011 = -12 (۸ بیت). جمع: نقلی خروجی MSB به نتیجه اضافه شود.
- مکمل دو: $-x = 2^n - x$ = مکمل یک + 1؛ 11110100 = -12. میان‌بر: از راست تا اولین 1 کپی، بقیه معکوس. جمع: نقلی دور ریخته شود.
- تفریق: A - B = A + (-B).
- سریع ۸ بیتی:

-1=FF -2=FE -8=F8 -10=F6 -16=F0 -100=9C -127=81 -128=80؛
۱۶ بیتی: -1=FFFF -200=FF38.

- کسری: SM 01011.011 ⇒ 11011.011؛ 1s ⇒ 00111.101 → 11000.010؛ 2s ⇒ 10110.101 → 01001.011 (افزودن 1 به LSB).

مثال‌های ۴ بیتی اسلاید

وضعیت	Result	Binary	Op	Sys
OK	-7	1101+1010=1 0111→1000	-2+-5	1s
OK	-0	0101+1010=1111	+5+-5	1s
سرریز	+5	1100+1000=1 0100→0101	-3+-7	1s
OK	-8	1110+1010=1 1000	-2+-6	2s
OK	+3	0110+1101=1 0011	+6+-3	2s
سرریز (Cin0/Cout1)	+7	1101+1010=1 0111	-3+-6	2s
سرریز (Cin1/Cout0)	-5	0101+0110=1011	+5+6	2s
OK	+5	0110+1111=1 0101	6-1	2s
سرریز	+7	1011+1100=1 0111	-5-4	2s

اثبات‌ها (تکلیف): جمع‌کنندهٔ دودویی

مکمل یک: افزودن نقلی چرخشی = کم کردن 2^n-1. حالت‌ها: (+,+) بی‌نقلی ✓؛ (+,-): R=(A-|B|)+(2^n-1)؛ اگر A>|B| نقلی ⇒ A-|B| ✓؛ وگرنه (2^n-1)-(|B|-A) ✓؛ (-,-): (2^n-1)-(|A|+|B|) = 2^n+2^n-|A|-|B|-2 = 2^n-|A|-|B|-1، نقلی ⇒ (2^n-1)-(|A|+|B|) ✓.

مکمل دو: جمع‌کننده $(a+b) \bmod 2^n$. (+,-): (A-|B|)+2^n ✓ هر دو حالت؛ (-,-): (|A|+|B|)-2^n ⇒ دور ریختن ⇒ 2^n+2^n-(|A|+|B|). ✓ سرریز: (+,+) ⇒ Cout=0، نتیجهٔ منفی یعنی Cin=1؛ (-,-) ⇒ Cout=1، نتیجهٔ مثبت یعنی Cin=0.

نمایش افزونی (Excess-K)

کد = مقدار + K؛ توزیع یکنواخت با $K = 2^{n-1}$ یا $2^{n-1}-1$ (۴ بیت: Excess-8 یا 7؛ ۳ بیت: 3=111 … -4=000 ⇒ Excess-4). K دیگر ⇒ ناهمسان (Excess-5 در ۴ بیت: -5…+10). Excess-2^{n-1} = مکمل دو با MSB معکوس. Excess-127:
01101011 → 6، -20 → 10000101.

۱۳ ممیز ثابت و ممیز شناور

ممیز ثابت

تعداد بیت‌های صحیح/کسری ثابت؛ ممیز فرضی. ۸ بیت (۶.۲) مکمل دو:
26.75 = 011010.11، -1.25 = 111110.11؛ بازه 31.75 … -32 با گام 0.25؛
-5.5 = 111010.10. عیب: بازهٔ محدود.

قالب سفارشی ۱۲ بیتی اسلاید

S1	M7	E4

$V = (-1)^S \times 0.M \times 2^{E-8}$ (مانتیس SM، نما Excess-8)

کدگذاری: ۱) به دودویی ۲) نرمال کن $0.1xxx \times 2^e$ ۳) E = e + bias ۴) M = ۷ بیت بعد از ممیز (بقیه **بریده** می‌شود) ۵) S.

Value	S M E	مورد
+127	0 1111111 1111	بزرگ‌ترین
-127	1 1111111 1111	کوچک‌ترین
-2^{-15}	1 0000001 0000	بزرگ‌ترین منفی
نرمال: 2^{-15} (2^{-9}-2)	0 0000001 0000	کوچک‌ترین مثبت
-3.625 = -0.11101×2²	1 1110100 1010	نرمال
25.25 !!	0 1100101 1101	25.375 = 0.11001011×2⁵
	0 1010110 1100 / 1 1100010 1100	10.75 / -12.25
	1 1011100 1101 / 0 1011100 1101	-23.0 / 23.0
0.1111111×2⁻²	0 1111111 0110	0.248046875
0.0×2^{-8}	0 0000000 0000	صفر (درست)

جمع/تفریق، ضرب و تقسیم

1. **هم‌ترازی:** مانتیس عدد با نمای کوچک‌تر را به اندازهٔ اختلاف نماها به راست شیفت بده (بیت‌ها از دست می‌روند).
2. جمع/تفریق مانتیس‌ها.
3. نرمال‌سازی: سرریز مانتیس ⇒ شیفت راست و E+1 (LSB بریده)؛ صفر پیشرو ⇒ شیفت چپ و E-1؛ مانتیس صفر ⇒ **E = 0**.

ضرب: $S=S_1 \oplus S_2$, $M=M_1 \times M_2$, $E=E_1+E_2-bias$ (بایاس اضافه). **تقسیم:** $E=E_1-E_2+bias$ (بایاس حذف‌شده).

```
3 x 2.5:  3 = 0 1100000 1010 ; 2.5 = 0 1010000 1010
M = 0.11 x 0.101 = 0.01111 ; E = 1010+1010-1000 = 1100
normalize: 0.1111 x 2^-1 → E = 1011
result = 0 1111000 1011 = 0.1111 x 2^3 = 7.5
```

مثال‌های جمع اسلاید (S M E)

```
11.5 + 23.25
11.5  = 0 1011100 1100   (0.10111 x 2^4)
23.25 = 0 1011101 1101   (0.1011101 x 2^5)
align 11.5 → 0 0101110 1101  (shift 1)
0101110 + 1011101 = 10001011  (8 bits!)
truncate LSB, E+1 → 0 1000101 1110 = 34.5 (not 34.75)
0.6015625 + 12.375  (exact 12.9765625)
0.6015625 = 0 1001101 1000   (0.1001101 x 2^0)
12.375    = 0 1100011 1100   (0.1100011 x 2^4)
align: shift 4 → 0 0000100 1100 (lost 1101)
0000100 + 1100011 = 1100111 → 12.875
(slide used E=0111 by mistake → 12.625)
```

پیام: در جمع/تفریق ممیز شناور عدد کوچک‌تر فدای بزرگ‌تر می‌شود؛ هرچه اختلاف نما بیشتر، جریمه بیشتر.

```
R0=10.75 R1=-12.25 R2=-23.0 R3=0.248046875
FSUB R0,R1 → R0 = 0 1011100 1101 = 23.0
FADD R2,R0 → R2 = 0 0000000 1101 = 0.0 x 2^5
FADD R3,R2 → R3 shifted 7 bits → 0.0 !!!
fix: zero = 0 0000000 0000 = 0.0 x 2^-8
```

پرسش‌های مفهومی

- **چرا نرمال‌سازی؟** ۱) بهترین استفاده از بیت‌های مانتیس (دقت بیشینه) ۲) یکتایی نمایش هر عدد.
- **چرا کوچک‌ترین نما برای صفر؟** کمینه کردن خطای محاسبه (در هم‌ترازی، مانتیس صفر شیفت می‌خورد نه عدد دیگر).
- **مزایای نمای بایاس‌دار:** ۱) صفر = الگوی تمام‌صفر ۲) مقایسهٔ نماها ساده (بدون علامت) ۳) آزمون صفر با مدار اعداد صحیح ۴) بدون کد عمل جدا برای پرش صفر/ناصفر ممیز شناور. **معایب:** بایاس اضافه در ضرب، بایاس حذف‌شده در تقسیم.

۱۴ استاندارد IEEE 754

S	E	F

$V = (-1)^S \times 1.F \times 2^{E-bias}$؛ **Single**: 1+8+23، bias=127؛ **Double**: 1+11+52، bias=1023؛ بیت ۱ پنهان (ذخیره نمی‌شود).

کدگذاری: ۱) دودویی ۲) $1.xxx \times 2^e$ ۳) E = e+127 (۸ بیت) ۴) F = بیت‌های بعد از ممیز، با صفر پر/گرد شود ۵) گروه‌های ۴ بیتی ⇐ هگز. **رمزگشایی:** برعکس؛ اگر E=0 ⇐ غیرنرمال $0.F \times 2^{-126}$.

```
-6.5 = -110.1 = -1.101 x 2^2 ; E = 2+127 = 129
     = 1 10000001 10100000000000000000000 = C0D00000
110.1      → 1.101   x 2^2   (store 101)
0.00101101 → 1.01101 x 2^-3  (store 01101)
C2F00000 = 1 10000101 111000.. = -1.111 x 2^6 = -120
85.125 = 1.010101001 x 2^6 → 42AA4000
```

Value	Hex	S | E(8) | F(23)
+0	00000000	0 00000000 00000000000…
-0	80000000	1 00000000 00000000000…
1.0	3F800000	0 01111111 00000000000…
-1.0	BF800000	1 01111111 00000000000…
2.0	40000000	0 10000000 00000000000…
0.5	3F000000	0 01111110 00000000000…
0.1 (≈)	3DCCCCCD	0 01111011 10011001100…
0.15625	3E200000	0 01111100 01000000000…
10.75	412C0000	0 10000010 01011000000…
25.0	41C80000	0 10000011 10010000000…
85.125	42AA4000	0 10000101 01010100100…
-6.5	C0D00000	1 10000001 10100000000…
-120.0	C2F00000	1 10000101 11100000000…
+∞	7F800000	0 11111111 00000000000…
-∞	FF800000	1 11111111 00000000000…
min normal 2^{-126}	00800000	0 00000001 00000000000…
min denormal 2^{-149}	00000001	0 00000000 00000000000…
max ≈3.4028×10^{38}	7F7FFFFF	0 11111110 11111111111…
NaN (example)	7FC00000	0 11111111 10000000000…

Double: 3FF0000000000000 = 1.0، C01A000000000000 = -6.5،
0.1 ≈ 3FB9999999999999A.

	Denormalized	Normalized	≈ Decimal
Single	$\pm2^{-149}$ … $(1-2^{-23})\times2^{-126}$	$\pm2^{-126}$ … $(2-2^{-23})\times2^{127}$	$\pm10^{-44.85}$ … $10^{38.53}$
Double	$\pm2^{-1074}$ … $(1-2^{-52})\times2^{-1022}$	$\pm2^{-1022}$ … $(2-2^{-52})\times2^{1023}$	$\pm10^{-323.3}$ … $10^{308.3}$

Operation	Result
n ÷ ±∞	0
±∞×±∞, ±finite×±∞	±∞
±nonZero ÷ ±0	±∞
∞+∞, ∞-(-∞)	+∞
-∞-∞, -∞+(-∞)	-∞
∞-∞, -∞+∞	NaN
±0÷±0, ±∞÷±∞, ±∞×0	NaN
NaN == NaN	false

E	F	Value
00…0	0	±0 (برابر)
00…0	≠0	$\pm0.F\times2^{1-bias}$
00…1–11…0	any	$\pm1.F\times2^{E-bias}$
11…1	0	±∞
11…1	≠0	NaN

۱۵ کدهای دهدهی و کاراکتری

BCD: هر رقم ۴ بیت. 0010 0101 0110 1001 1000 0000 0111 → 7089652.

دسته‌بندی: وزن‌دار (همه‌مثبت/مختلط) و بی‌وزن؛ ۴ بیتی، ۵ بیتی، بیش از ۵ بیت.

d	8421	Ex-3	5421	84-2-1	7421	5311	631-1	2421
0	0000	0011	0000	0000	0000	0000	0011	0000
1	0001	0100	0001	0111	0001	0001	0010	0001
2	0010	0101	0010	0110	0010	0011	0101	0010
3	0011	0110	0011	0101	0011	0100	0111	0011
4	0100	0111	0100	0100	0100	0101	0110	0100
5	0101	1000	0101	1011	0101	1000	1001	1011
6	0110	1001	0110	1010	0110	1001	1000	1100
7	0111	1010	0111	1001	1000	1011	1010	1101
8	1000	1011	1011	1000	1001	1100	1101	1110
9	1001	1100	1100	1111	1010	1101	1100	1111

d	2-of-5	63210	Shift-cnt	86421	51111	50 43210	543210
0	00011	00001	00000	00000	00000	01 00001	000001
1	00101	00011	00001	00001	00001	01 00010	000010
2	00110	00101	00011	00010	00011	01 00100	000100
3	01001	01001	00111	00011	00111	01 01000	001000
4	01010	01010	01111	00100	01111	01 10000	010000
5	01100	01100	11111	00101	10000	10 00001	100001
6	10001	10001	11110	01000	11000	10 00010	100010
7	10010	10010	11100	01001	11100	10 00100	100100
8	10100	10100	11000	10000	11110	10 01000	101000
9	11000	11000	10000	10001	11111	10 10000	110000

9876543210 (one-hot): رقم d ⇐ فقط بیت d یک است (۱۰ بیت).

- **خودمکمل:** مکمل ۹ با NOT بیتی؛ **Excess-3، 631-1، 2421، 51111** (و 84-2-1). شرط لازم برای کد وزن‌دار: جمع وزن‌ها = ۹.
- **Excess-3 = BCD + 3**؛ 0110 1100 1000 → 395.
- **جمع BCD:** اگر رقم > 9 یا نقلی داد ⇒ +6 (0110):
  42 = 0100 0010 → 0011 1100 :27+15.
- **out-of-5-2:** دقیقاً دو بیت ۱ ⇒ آشکارسازی خطای تک‌بیتی.
- **گری:** اعداد متوالی یک بیت اختلاف؛ $g_i=b_{i+1}\oplus b_i$، $g_{n-1}=b_{n-1}$؛ برعکس $b_i=b_{i+1}\oplus g_i$.

n	Binary	Gray
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

n	Binary	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

ASCII (۷ بیتی) — ستون = ۳ بیت پرارزش، سطر = ۴ بیت کم‌ارزش

low\high	0 000	1 001	2 010	3 011	4 100	5 101	6 110	7 111
0 0000	NUL	DLE	SP	0	@	P	`	p
1 0001	SOH	DC1	!	1	A	Q	a	q
2 0010	STX	DC2	"	2	B	R	b	r
3 0011	ETX	DC3	#	3	C	S	c	s
4 0100	EOT	DC4	$	4	D	T	d	t
5 0101	ENQ	NAK	%	5	E	U	e	u
6 0110	ACK	SYN	&	6	F	V	f	v
7 0111	BEL	ETB	'	7	G	W	g	w
8 1000	BS	CAN	(	8	H	X	h	x
9 1001	HT	EM	)	9	I	Y	i	y
A 1010	LF	SUB	*	:	J	Z	j	z
B 1011	VT	ESC	+	;	K	[	k	{
C 1100	FF	FS	,	<	L	\	l	|
D 1101	CR	GS	-	=	M	]	m	}
E 1110	SO	RS	.	>	N	^	n	~
F 1111	SI	US	/	?	O	_	o	DEL

- 'A'=41h=65, 'a'=61h=97, SP=20h, LF=0Ah, CR=0Dh, DEL=7Fh ،'0'=30h.
- تفاوت حرف کوچک/بزرگ = بیت ۵ (20h): کوچک OR 20h، بزرگ AND DFh، تعویض XOR 20h. رقم ← مقدار: 30h- یا AND 0Fh.
- **ASCII**: ۸ بیت (۷ + توازن) در PCها. توازن زوج: 'A'→01000001، 'C'→11000011.
- **EBCDIC**: ۸ بیتی **Unicode**. IBM: همهٔ زبان‌ها، UTF-8/16/32.

Code point	UTF-8 bytes
U+0000–007F	0xxxxxxx
U+0080–07FF	110xxxxx 10xxxxxx
U+0800–FFFF	1110xxxx 10xxxxxx 10xxxxxx
U+10000–10FFFF	11110xxx 10xxxxxx ×3

«س» = U+0633 ⇒ 11011000 10110011 :D8 B3؛ «CSL» = 43 53 4C.

۱۶ آدرس‌دهی نسبی، قطعه‌ای، صفحه‌ای و چندکاربره

- **نسبی:** بیشتر پرش‌ها نزدیک‌اند ⇒ به‌جای آدرس کامل، جابه‌جایی کوتاه (بیت کمتر). اسمبلر d را از آدرس مقصد و محل فعلی می‌سازد. کد فقط با پرش نسبی **مستقل از مکان** است. jcxz loop_addr (8086، d هشت بیتی).
- **قطعه‌ای:** آدرس فیزیکی = آدرس منطقی + ثبات قطعه. **مثبت:** OS (پیونددهنده/بارکننده) دردسر کمتر، کد تغییر نمی‌کند. **منفی:** یک جمع برای هر دسترسی ⇒ اجرای کندتر. (8086: seg×16 + off)
- **صفحه‌ای:** صفحه 2^P کلمه، حافظه 2^M ⇒ ثبات صفحه M-P بیت، آدرس منطقی P بیت، 2^{M-P} صفحه. **مثبت:** بدون جمع (فقط الحاق) ⇒ مزایای قطعه‌ای بدون عیبش. **منفی:** شروع فقط در مضرب 2^P، حداقل طول 2^P، برنامهٔ بزرگ‌تر ⇒ چند صفحه، مدیریت دشوار.

- صفحه‌ای: حافظه 2^{16}، صفحه 1024، LA=0A3h ⇒ PR=000101، 14A3h = 0010100011‖000101.
- حافظه 2^{20}، صفحهٔ ۸ PR ⇒ 4K بیت، ۱۲ LA بیت، ۲۵۶ صفحه؛ برنامهٔ ۳ ⇒ 10K صفحه، ۲K هدر.
- قطعه‌ای: 0003B234h = 00001234h + 0003A000h؛ صفحهٔ 64K با PR=0003h ⇒ 00031234h.
- load (p) با p=16Fh و 17Fh=(p) ⇒ دسترسی به 5016Fh سپس SR=00050000h، 5017Fh.

پشتیبانی چندکاربره

- هر کاربر هنگام login بخشی از حافظه (بسته به سطح کاربر) می‌گیرد و هنگام logout آزاد می‌شود؛ CPU با **تسهیم زمانی** میان کاربران فعال تقسیم می‌شود.
- OS جدول کاربران فعال دارد: User ID، شروع/پایان قطعه، حق دسترسی/اولویت، اشاره‌گر به Context، ...
- **Context** = ثبات‌ها (قطعه/صفحه، PC، وضعیت، ...)؛ ذخیرهٔ متن کاربر قبلی و بار متن بعدی = **تعویض متن**. زمان‌بر ⇒ برش زمانی طولانی‌تر = بهرهٔ بهتر CPU. GPUها: فایل ثبات بزرگ ⇒ تعویض متن بدون تأخیر.
- حفاظت: بررسی آدرس با شروع/پایان قطعه و حق دسترسی.

۱۷ فون‌نویمان، فلین و ابررایانه‌ها

- **فون‌نویمان (۱۹۴۵):** برنامه و داده در **یک حافظه**؛ CPU = CU + ALU؛ ورودی/خروجی؛ چرخهٔ واکشی–رمزگشایی–اجرا. گلوگاه: یک مسیر CPU–حافظه، یک دستور در لحظه ⇒ ساختارهای پیشرفته برای بهبود کارایی.
- **CU**: واکشی، رمزگشایی، فرستادن سیگنال کنترلی. **ALU**: عملیات حسابی و منطقی.

دسته	تعریف	نمونه
SISD	یک IS، یک DS	همان فون‌نویمان، تک‌هسته‌ای
SIMD	یک CU، n واحد PU	برداری (Cray-1)، GPU، SSE/AVX؛ بیشتر ابررایانه‌های اولیه
MISD	چند IS روی یک DS	خط لوله، سیستولیک (محل اختلاف؛ «وجود ندارد»)؛ خط لوله مقرون‌به‌صرفه‌ترین
MIMD	چند CU/PU/MU مستقل	چندکامپیوتری؛ تقریباً همهٔ ابررایانه‌های فعلی، چندهسته‌ای

سه دستهٔ SIMD/MISD/MIMD = «ساختارهای پیشرفته». IS = جریان دستور، DS = جریان داده.

سیستم	سال	نوع	مشخصات کلیدی
Cray-1	1975	SIMD	160 MFLOPS، برداری، 8MB
Cray T3E	1995	MIMD	تا 2048 PE، 40–2 TF، Alpha 21164A، 1350 MF/PE، 10GB–1TB، جنبرهٔ ۳بعدی
Earth Simulator	2002	Multi-SIMD	640 گره × 8 برداری = 5120، 8 GF/پردازنده ⇒ 40 TF، $10TB، 350M
BlueGene	2004–07	MIMD	360 TF، 65536 گره، جنبرهٔ ۳بعدی، $130M
Cray X1	—	SIMD–MIMD	T3E + PVP؛ 4 × 12.8 GF/MSP=گره؛ NUMA تا 1024 گره
Roadrunner	2008	MIMD	اولین پتافلاپس: 1.33 PF، 12960 Cell (9 هسته)، 133M، $3.9MW
LineShine	2026	MIMD	2.735≈ EF، 13,789,440 هسته، 42.2MW، 13PB، $≈1B

FLOPS = عمل ممیز شناور در ثانیه؛ TOP500 هر ۶ ماه؛ K 10^3, M 10^6, G 10^9, T 10^{12}, P 10^{15}, E 10^{18}. LineShine، El (ژوئن ۲۰۲۶): ده رتبهٔ اول Capitan، Frontier، Aurora، JUPITER، Eagle، HPC6، Fugaku، Alps، LUMI. مغز: ≈1.2 PF، 20W، 1400cm³ (≈۱۱۵۰ برابر کم‌مصرف‌تر)؛ K-Computer (10 PF) یک ثانیه از مغز را در ۴۰ دقیقه.

۱۸ پرسش‌های تشریحی با پاسخ کوتاه

1. **مدل برنامهٔ ذخیره‌شده؟** برنامه و داده در یک حافظه؛ اجرای برنامهٔ دیگر فقط با بار کردن آن (بدون سیم‌کشی دوباره).
2. **چرا زبان اسمبلی؟** حافظه و زمان کمتر، کارهای وابسته به سخت‌افزار، کارهای حساس به زمان، ISR و برنامه‌های مقیم؛ (شکستن قفل و ویروس). دانش رابط با OS/BIOS، نمایش داده، دسترسی به دستور/داده/I/O.
3. **چرا اسمبلر دوگذره است؟** ارجاع رو به جلو؛ گذر ۱ جدول نمادها، گذر ۲ کد.
4. **اگر بعد از حلقه به OS برنگردیم؟** PC به داده‌ها می‌رسد و CPU آن‌ها را دستور فرض می‌کند ⇒ رفتار نامعلوم.
5. **مزایا و معایب کد خودتغییرده؟** خوب برای رمزنگاری و قفل نرم‌افزاری؛ بد از نظر مهندسی نرم‌افزار و نتیجهٔ غلط در اجرای چندباره.
6. **چرا آدرس‌دهی نسبی؟** پرش‌ها نزدیک‌اند ⇒ فیلد کوتاه‌تر، دستور کوتاه‌تر؛ کد مستقل از مکان.
7. **قطعه‌ای در برابر صفحه‌ای؟** قطعه: جمع (کند) ولی انعطاف؛ صفحه: الحاق (سریع) ولی مرز و طول ثابت.
8. **مشکل بارکنندهٔ جابه‌جاکننده؟** تشخیص آدرس از عدد در فیلدهای بی‌واسطه و اشاره‌گرهای داده.
9. **تعویض متن چیست و چرا مهم است؟** ذخیره/بار ثبات‌های کاربر؛ هزینه‌بر ⇒ برش زمانی بزرگ‌تر.
10. **چرا مکمل دو رایج است؟** یک صفر، جمع‌کنندهٔ ساده (بی‌نقلی چرخشی)، تفریق = جمع با مکمل.
11. **چرا جمع SM سخت است؟** مقایسهٔ اندازه‌ها، جمع‌کننده و تفریق‌کننده، انتخاب علامت.
12. **چرا 0.1 دقیق نیست؟** بسط دودویی متناوب است؛ اعداد حقیقی تقریبی‌اند چون بیت‌ها محدودند.
13. **نقش ثبات‌های MAR/MBR/IR/PC؟** بخش ۴.
14. **MISD چرا محل اختلاف است؟** بعضی می‌گویند ماشین واقعی ندارد؛ بعضی خط لوله و سیستولیک را MISD می‌دانند.
15. **نمادگذاری لهستانی چرا؟** بدون پرانتز/اولویت، مستقیم روی پشته محاسبه می‌شود.
16. **فرق Register File و Isolated؟** یکی در هر لحظه / دسترسی موازی.
17. **فایدهٔ نگهبان؟** حذف بررسی مرز در حلقه (کد ساده‌تر و سریع‌تر).

۱۹ تله‌های امتحانی و خطاهای اسلاید

- **jpos:** در ماشین‌های ۳، ۴ و تک‌آدرسه 0 ≤؛ در ماشین ۷ 0 ≥.
- جابه‌جایی نسبی از **دستور بعدی** حساب می‌شود، نه از خود پرش.
- حافظهٔ نیبلی (ماشین ۴ و تک‌آدرسه): آدرس‌ها بر حسب ۴ بیت؛ هر رقم هگز یک واحد؛ کلمه = ۴ واحد.
- ماشین ۲ پرش ندارد (باز کردن حلقه)؛ ماشین ۳ بی‌واسطه و پرش غیرشرطی ندارد (ثابت‌ها در حافظه).
- MBR = پهنای دسترسی، نه واحد آدرس‌پذیر؛ IR = بلندترین قالب؛ PC = MAR.
- pop داده را پاک نمی‌کند؛ در ماشین ۶ اول شرط بعد آدرس مقصد push می‌شود؛ sub = پایینی - بالایی.
- ثابت منفی در دستور: مکمل دو با طول فیلد (FFFF = -1 در ۱۶ بیت).
- در مکمل دو، $-(2^{n-1})$ قابل نمایش نیست (سرریز). جمع دو عدد ناهم‌علامت هرگز سرریز نمی‌کند.
- FP: هم‌ترازی به سمت نمای بزرگ‌تر؛ مانتیس بریده می‌شود (نه گرد) در قالب اسلاید؛ نتیجهٔ صفر ⇒ نما صفر.
- IEEE: نمای تمام‌صفر یعنی 2^{-126} و F.0 (نه F.1 و نه 2^{-127}).
- جمع +6 :BCD برای رقم > 9 یا نقلی.
- کلمهٔ ناهم‌تراز که از مرز سطر بگذرد ⇒ دو دسترسی.

خطاهای اسلاید (در جزوه اصلاح شده‌اند): کد jn ماشین ۶ (نوشته‌شده 08 = j؛ درست 0B)؛ load temp1 در 12Ah ماشین ۷ (01 ⇐ درست 00)؛ نمای 0.6015625 (0111 ⇐ درست 1000، نتیجه 12.875)؛ فیبوناچی ماشین ۶ با counter=10 جمع ۲۲ جمله (=28656) است، برای ۲۰ جمله (=10945) counter=9؛ سرریز نگهبان مرتب‌سازی؛ UFT ⇐ UTF.

۲۰ داده‌های مرجع

فیبوناچی $\Sigma F_{0..n-1} = F_{n+1} - 1$؛ $F_n = (-1)^n F_{n-1} - 1$؛ $\Sigma_{0..n}(-1)^i F_i = (-1)^n F_{n-1}$

i	F_i	$\Sigma F_{0..i}$	i	F_i	$\Sigma F_{0..i}$
0	0	0	1	1	1
2	1	2	3	2	4
4	3	7	5	5	12
6	8	20	7	13	33
8	21	54	9	34	88
10	55	143	11	89	232
12	144	376	13	233	609
14	377	986	15	610	1596
16	987	2583	17	1597	4180
18	2584	6764	19	4181	10945
20	6765	17710	21	10946	28656
22	17711	46367	23	28657	75024
24	46368	121392	25	75025	196417

اعداد اول

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, ...

صدمین عدد اول = **541**؛ مجموع ۱۰ / ۲۰ / ۵۰ / ۱۰۰ عدد اول:
129 / 639 / 5117 / **24133**